

Gestione dello sbilanciamento tra le classi

Laboratorio di Intelligenza Artificiale

Vincenzo Bonnici

Corso di Laurea Magistrale in Scienze Informatiche

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Università degli Studi di Parma

2025-2026

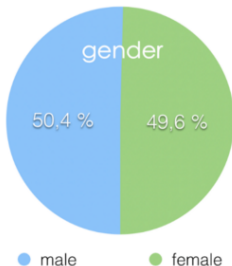
Il problema dello **sbilanciamento** è presente in molte delle applicazioni pratiche dell'apprendimento automatico supervisionato (e non).

Il problema nasce quando si deve riconoscere una classe per cui si hanno **pochi campioni** nel data set, ovvero la classe è rara.

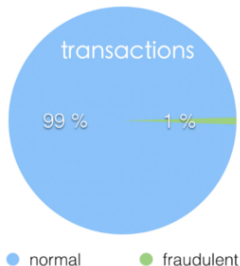
A causa della disparità tra le numerosità della classi, un algoritmo di **apprendimento automatico** tenderà a riconoscere meglio la classe (o le classi) con maggiori campioni e ad avere delle **prestazioni molto ridotte** sulla classe (o le classi) rara.

In realtà il problema è più complesso di così. Infatti, potremmo non accorgerci di tali prestazioni ridotte se la **misura di qualità** se stiamo utilizzando (es. accuratezza) **non è adeguata** ad un data set sbilanciato.

Balanced Dataset



Unbalanced Dataset



Esso è più comunque di quanto non si pensi. Queste, ad esempio, sono delle applicazioni tipiche in cui il problema dello sbilanciamento non è dovuto semplicemente ad un problema di campionamento ma è **intrinseco** nella particolare applicazione: rilevamento di frodi; rilevamento di anomalie; diagnosi di malattie rare; fuoriuscite di petrolio; rilevamento facciale; etc ...

L'algoritmo riceve molti più esempi da una classe, spingendolo a essere sbilanciato verso quella particolare classe. Non impara cosa rende l'altra classe "diversa" e non riesce a comprendere i **pattern sottostanti** che ci consentono di distinguere le classi.

L'algoritmo apprende che una data classe è più comune, rendendo "naturale" una maggiore **tendenza** verso di essa. L'algoritmo è quindi incline a **sovra-adattarsi** alla classe maggioritaria. Solo prevedendo la classe maggioritaria, i modelli otterrebbero un punteggio elevato nelle loro funzioni di costo.

In questi casi appare il **paradosso dell'accuratezza**. Se la classe A è il 99% dei campioni iniziali, il classificatore sceglierà sempre A come risposta e l'accuratezza sarà comunque di circa il 99%. Una soluzione è usare precisione e recupero (recall), ma anche cambiato misura non aiutiamo il classificatore nelle **istanze problematiche**.

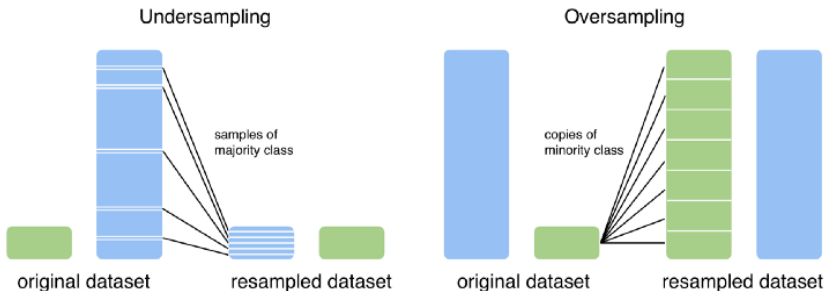
Prima di decidere di ri-campionare il data set facciamo le seguenti osservazioni:

- possiamo **collezionare più dati**? Se sì, conviene sempre farlo perché i dati artificiali non possono mai sostituire i dati reali.
- ricordiamoci di utilizzare un **adeguato algoritmo** di apprendimento automatico. Alcuni algoritmi sono più robusti degli altri ma la loro scelta deve essere fatta anche in base agli obiettivi della classificazione.
- alcuni modelli di apprendimento possono **penalizzare** maggiormente gli errori di classificazione per le classi minoritarie, eliminando così il problema di ri-campionare il data set.
- alcune volte è meglio **cambiare approccio**. Ad esempio, se dobbiamo riconoscere delle frodi, forse è meglio utilizzare un algoritmo di rinascimento degli outlier.

Ri-campionamento

Una delle soluzioni più adottate per gestire i data set sbilanciati è il **ri-campionamento**. Esso si divide in due tipologie principali:

- **sotto-campionamento** (undersampling): riduciamo i campioni della classe più frequente
- **sovra-campionamento** (oversampling): aumentiamo in modo artificiale i campioni della classe meno frequente

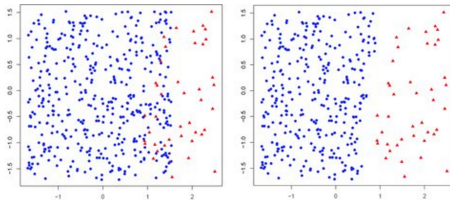


anche se non sono le uniche strategie possibili

Sotto-campionamento

Nel **sotto-campionamento** facciamo sì che la classe (o le classi) maggioritarie siano ridotte in modo da ottenere una numerosità che sia **compatibile** con la numerosità delle classi sotto-rappresentate.

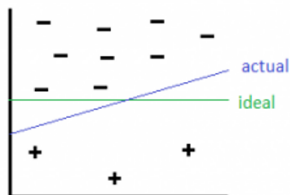
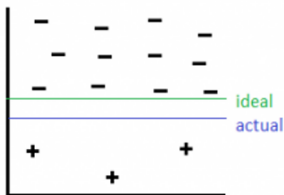
La classe maggioritaria viene ri-campionata in modo casuale, stando attenti a campionare **uniformemente** su tutte le dimensioni.



Non necessariamente la classe ri-campionata deve avere la stessa numerosità della classe sotto-rappresentata. In alcuni casi, possiamo lasciare una certa **tolleranza** nella differenza di numerosità che può aiutare nel rendere il classificare più generalizzabile.

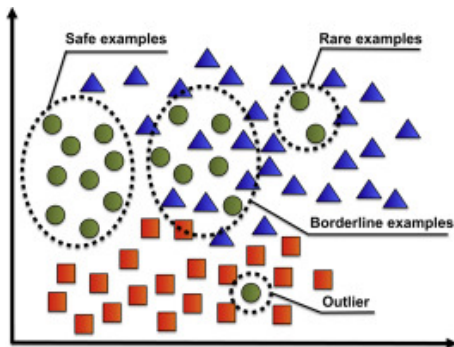
Sotto-campionamento

Ricordiamoci sempre che il **rischio** è quello di rimuovere alcuni punti importanti dalla classe di cui ne stiamo riducendo la numerosità. Questo comportamento può cambiare sensibilmente le prestazioni del modello computazionale perché va ad inferire la stima della **frontiera** e quindi dei pattern acquisiti per la corretta classificazione.



Sbilanciamento - sotto-campionamento

Stiamo sempre attenti a cosa eliminando.

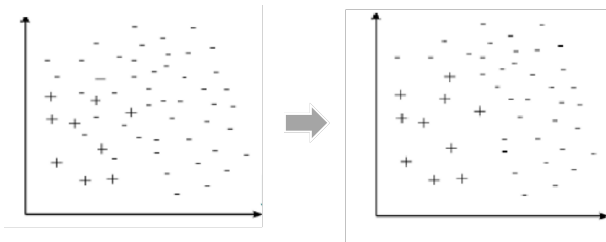


- rispetto al confine tra le classi, dove si trovano i punti eliminati?
- ha senso eliminare le istanze rare o gli outlier?

Sotto-campionamento - Collegamenti Tomek

Siano x_i e x_j due punti appartenenti a due **classi diverse** e sia $d(x_i, x_j)$ la distanza tra i due punti.

Allora, una coppia (x_i, x_j) è chiamata **collegamento (link) di Tomek** se non esiste nella collezione di punti un elemento x_l tale che $d(x_i, x_l) < d(x_i, x_j)$ oppure $d(x_j, x_l) < d(x_i, x_j)$, ovvero x_i è il punto più vicino ad x_j e viceversa.

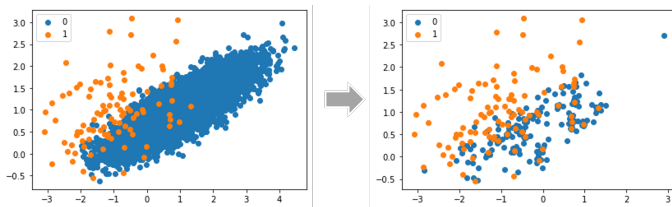


Rimuovendo i punti della classe maggioritaria che sono componenti di un Tomek link eliminiamo sia rumore che esempi borderline.

Sotto-campionamento - CNN

Il metodo CNN (Condensed Nearest Neighbor) cerca un sottoinsieme di una raccolta di campioni che non comporti alcuna perdita di prestazioni del modello, indicato anche come insieme minimo coerente.

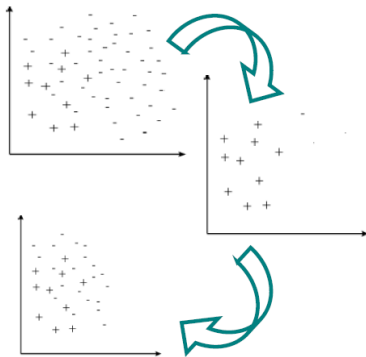
Si ottiene scorrendo i punti del data set e aggiungendoli al gruppo di output solo se non possono essere classificati correttamente dal contenuto corrente.



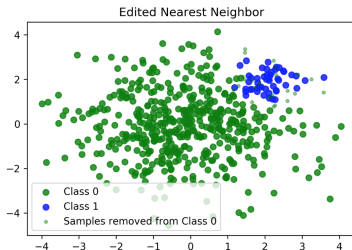
Sotto-campionamento - CNN

Passi:

- sia S il training set di partenza
- sia S' un insieme contenente tutti i punti dalle classe positiva e un elemento scelto a caso della classe negativa.
- si classifica tutto S usando un classificatore 1-NN che utilizza S'
- si muovono tutti i punti classificati erroneamente da S a S'



Edited nearest neighbors (ENN): usa un 3-NN. Quando un evento fa parte della classe maggioritaria ed è classificato erroneamente in base ai tre vicini più prossimi, viene rimosso. Quando un evento fa parte della classe di minoranza ed è classificato erroneamente in base ai suoi tre vicini più prossimi, i suoi vicini più vicini nella classe di maggioranza vengono rimossi.



Neighborhood cleaning rule (NCR): combina ENN con CNN. Rimuove i duplicati con CNN. Quindi, rimuove il rumore e gli elementi ambigui con ENN.

Sbilanciamento - sovra-campionamento

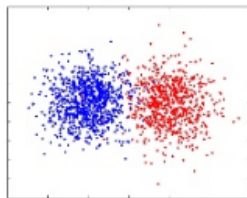
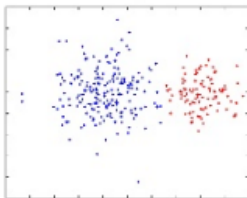
Nel **sovra-campionamento** aumentiamo la numerosità della classe sotto-rappresentata aggiungendo al data set dei **campioni artificiali** costruiti in base alle proprietà dei campioni originali, soprattutto allo loro disposizione nello spazio n -dimensionale.

Sampling: Rebalancing the dataset

Imbalanced Data

Under-sampling

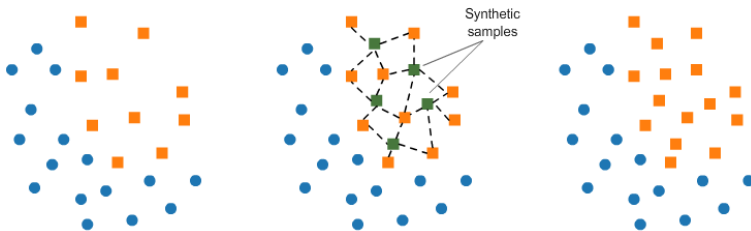
Over-sampling



Sovra-campionamento - SMOTE

SMOTE (synthetic minority over-sampling technique) è un metodo per la **generazione** di nuove istanze di dati, invece che replicare quelle esistenti, **interpolando** diversi esempi della classe minoritaria che giacciono insieme nello spazio delle feature.

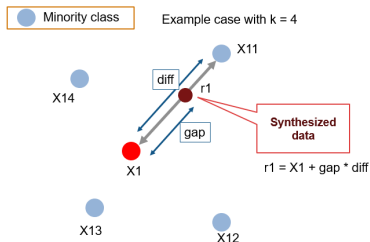
Per ogni classe minoritaria si introducono degli **esempi sintetici** che risiedono sui **segmenti** che uniscono tutti/alcuni k vicini sempre di classe minoritaria.



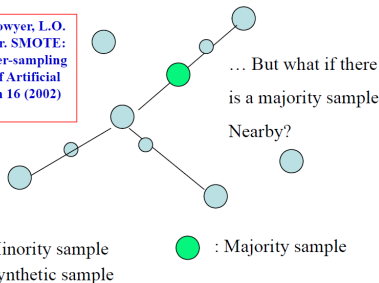
Sovra-campionamento - SMOTE

Per ogni esempio x_i della classe minoritaria:

- **trova** i suoi k vicini minoritari
- **scegli** a caso j dei suoi vicini (che dipende dal numero di esempi che si vogliono generare)
- **genera** in modo casuale esempi sintetici lungo i segmenti che uniscono x_i ai suoi j vicini scelti
 - **calcola** la differenza tra il vettore delle feature di x_i con il vettore delle feature del suo vicino.
 - **moltiplica** la differenza per un numero a caso tra 0 e 1
 - **aggiungi** il risultato al vettore delle feature di x_i



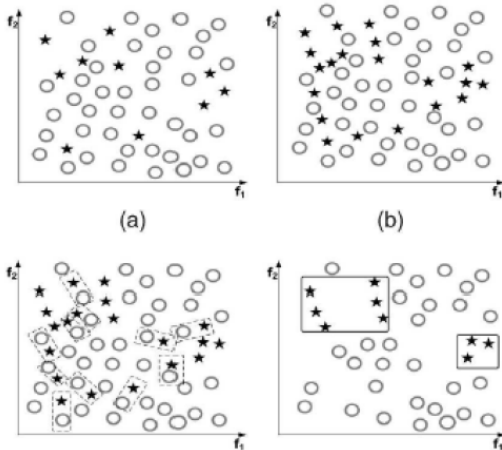
N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer. SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research* 16 (2002) 321-357



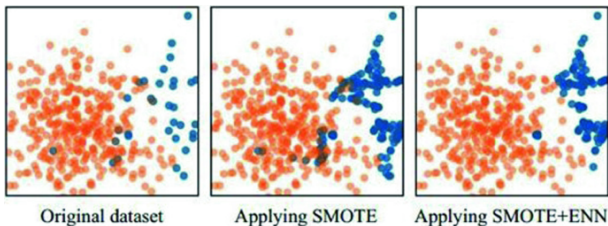
La procedura di SMOTE è intrinsecamente pericolosa poiché generalizza ciecamente l'area minoritaria senza riguardo alla classe maggioritaria. Questa strategia è particolarmente problematica nel caso di distribuzioni di classi altamente asimmetriche poiché, in tale casi, la **classe minoritaria è molto sparsa** di rispetto alla classe maggioritaria, risultando così in una maggiore possibilità di **mescolanza di classi**.

Sovra-campionamento - SMOTE + Tomek links

Una soluzione a tale problema può essere la combinazione di SMOTE con i Tomek link.



Lo possiamo anche combinare a ENN



- ENN rimuove qualsiasi esempio la cui etichetta di classe è diversa dalla classe di almeno due dei suoi vicini
- ENN rimuove più esempi dei Tomek link
- ENN rimuove più esempi da entrambe le classi

Possiamo anche modificare SMOTE per concentrarsi sui dati a **confine** tra le classi per rafforzare il confine verso l' "interno" della classe, ignorando però i punti rumorosi, ovvero gli **outlier**, definiti come punti i cui vicini sono **tutti** della classe maggioritaria.

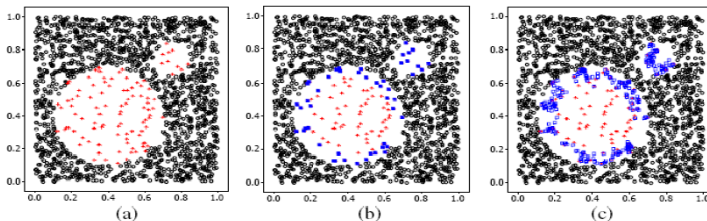
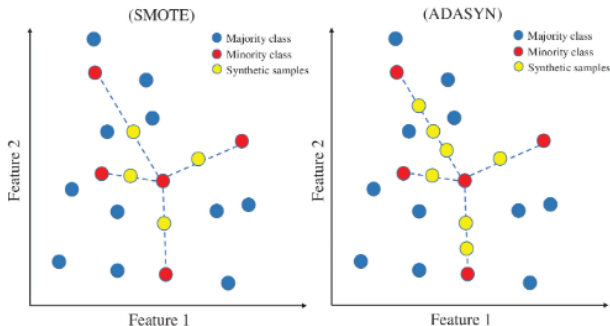


Fig. 1. (a) The original distribution of Circle data set. (b) The borderline minority examples (*solid squares*). (c) The borderline synthetic minority examples (*hollow squares*).

Sovra-campionamento - ADASYN

Per ciascuna delle osservazioni di minoranza trova prima l'**impurità del vicinato**, prendendo il rapporto tra le osservazioni di maggioranza nel vicinato e k .

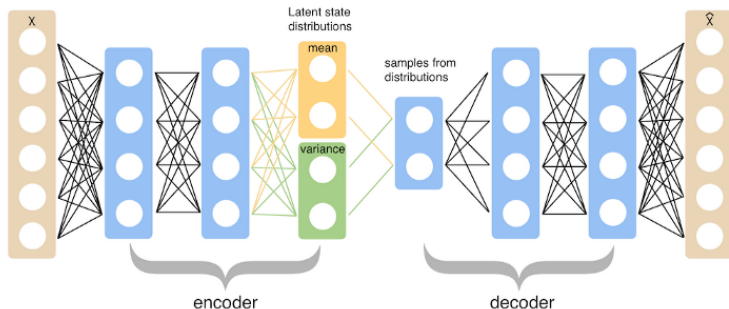
Tale rapporto di impurità viene convertito in una distribuzione di probabilità facendo la somma come 1. Quindi più alto è il rapporto vengono generati più punti sintetici per quel particolare punto.



Sovra-campionamento - VAE

Il metodo dei **variational autoencoders** (VAE) consente di esplorare le variazioni nei dati, non solo in modo casuale, ma nella direzione desiderata. In sintesi, una rete di autoencoder è composta da un codificatore (encoder) collegato a un decodificatore (decoder)

- Il **codificatore** registra i dati di input e li trasforma in una rappresentazione più piccola e densa.
- La rete del **decodificatore** utilizza questa rappresentazione e la riconverte nell'input originale.



Lo **spazio latente** a cui inviano i loro input ed è dove si trovano i loro vettori codificati potrebbe non essere continuo o consentire una facile interpolazione.

Nel caso dei VAE, i loro spazi latenti, in uscita dell'encoder, sono **continui per progettazione**, consentendo un campionamento e un'interpolazione casuali facili. Raggiungono questo risultato rendendo l'output del codificatore non un vettore di codifica, ma invece due vettori: μ e σ . Sono i parametri di una **distribuzione gaussiana** da cui si campiona, si ottiene il campione codificato e quindi lo si passa al decodificatore.

Gli autoencoder standard imparano a generare rappresentazioni compatte e **ricostruire correttamente** i loro input.

Lo **spazio latente** a cui inviano i loro input ed è dove si trovano i loro vettori codificati potrebbe non essere continuo o consentire una facile interpolazione.

Nel caso dei VAE, i loro spazi latenti, in uscita dell'encoder, sono **continui per progettazione**, consentendo un campionamento e un'interpolazione casuali facili. Raggiungono questo risultato rendendo l'output del codificatore non un vettore di codifica, ma invece due vettori: μ e σ . Sono i parametri di una **distribuzione gaussiana** da cui si campiona, si ottiene il campione codificato e quindi lo si passa al decodificatore.

Gli autoencoder standard imparano a generare rappresentazioni compatte e **ricostruire correttamente** i loro input.

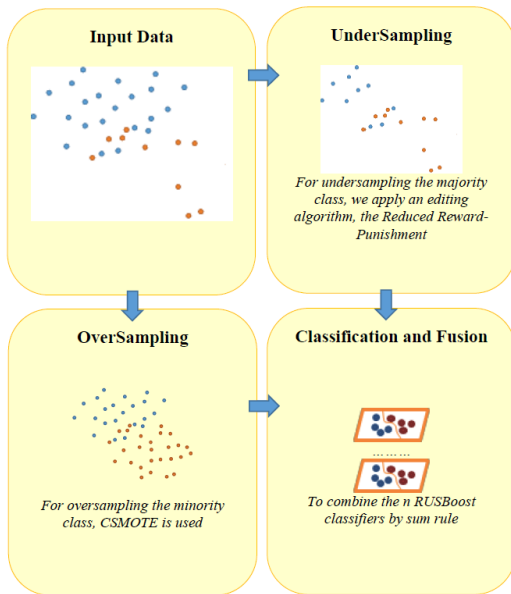
RUSBoost è un algoritmo che combina il campionamento dei dati e potenziamento.

È una variante di **SMOTEBoost** che crea nuovi esempi sintetici per la classe minoritaria utilizzando SMOTE, mentre RUSBoost realizza un **Random UnderSampling** (RUS) per rimuovere gli esempi dalla classe maggioritaria.

RUSBoost è molto simile ad **AdaBoost**: i pesi di ogni dato originale (esempio) sono inizializzati a $1/m$, dove m è il numero di esempi totali in training set, quindi per T volte (iterazioni) viene creato un training set applicando il Random UnderSampling sulla classe maggioritaria.

Rispetto a SMOTEBoost, questo algoritmo è meno computazionalmente complesso e dispendioso in termini di tempo.

Ri-campionamento - combinazione con HardEnsemble



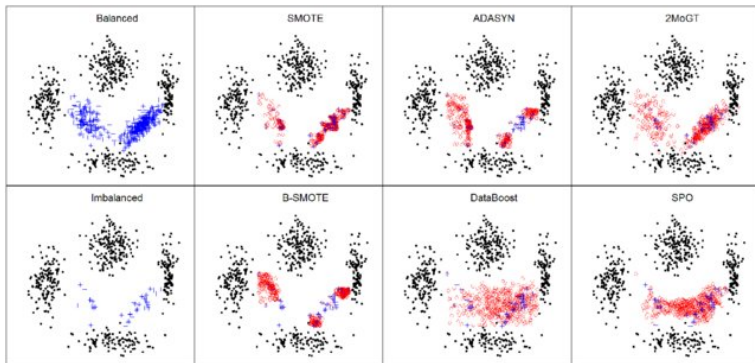
Osservazioni:

- sotto-campionare la classe maggioritaria può buttare via dati utili
- sovra-campionare la classe minoritaria può ridurre le prestazioni se gli esempi usati per la generazione sono degli outlier

Componenti:

- HardEnsemble contiene 50 **classificatori** che ricorrono a sovra-campionamento e 100 che utilizzano il sotto-campionamento che sono combinati usando una "sum rule". (il numero di classificatori è stato scelto empiricamente)
- i classificatori di **sovra-campionamento** sono addestrati su dati ottenuti tramite CSMOTE (vengono generati 5 volte i dati della classe maggioritaria)
- i classificatori di **sotto-campionamento** sono addestrati su dati ottenuti tramite Reduce-Reward Punishment (rimuove i dati che sono sulle regioni di sovrapposizione delle classi)

Sovra-campionamento - Valutazione critica



Una buona tecnica per capire se un metodo funziona bene potrebbe essere fare sotto-campionamento di un data set noto e vedere come il metodo ricostruisce tale data set.

Cao, H., Tan, V. Y., & Pang, J. Z. (2014). IEEE transactions on neural networks and learning systems, 25(12), 2226-2239.

Quando ri-campioniamo stiamo inevitabilmente alterando i dati di input che verranno usati per addestrare un classificatore e per procedere con altre analisi. Nel caso particolare del sovra-campionamento stiamo generando degli esempi che non sappiamo esistere in natura.

Una alternativa al ri-campionamento è il **cost-sensitive learning**, ovvero addestriamo assegnando dei costi specifici ai diversi tipi di errore.

La assegnazione dei costi consiste nel dare agli **errori** commessi per la classe minoritaria un peso maggiore rispetto agli errori commessi per la classe maggioritaria.

Questo, in effetti, rettifica il pregiudizio dato alla maggioranza classe da classificatori standard quando l'errore di addestramento corrisponde alla precisione semplice (non ponderata).

Cost-sensitive learning

Tradizionalmente, gli **algoritmi di apprendimento automatico** vengono addestrati su un set di dati e cercano di ridurre al minimo gli errori.

L'adattamento di un modello sui dati risolve un problema di ottimizzazione in cui cerchiamo esplicitamente di **ridurre al minimo l'errore**. È possibile utilizzare una serie di funzioni per calcolare l'errore di un modello sui dati di addestramento e il termine più generale è indicato come perdita. Cerchiamo di ridurre al minimo la perdita di un modello sui dati di addestramento, il che equivale a parlare di minimizzazione degli errori.

Nell'apprendimento sensibile ai costi, una **sanzione/peso** è associata a una previsione errata e viene definita "costo". Potremmo alternativamente riferirci all'inverso della sanzione come al "beneficio".

L'obiettivo dell'apprendimento sensibile ai costi è **ridurre al minimo il costo** di un modello **sul set di dati di addestramento**, in cui si presume che diversi tipi di errori di previsione abbiano un costo associato diverso e

Esiste uno stretto accoppiamento tra classificazione squilibrata e apprendimento sensibile ai costi. In particolare, un problema di apprendimento sbilanciato può essere affrontato utilizzando l'apprendimento sensibile ai costi.

Tuttavia, l'apprendimento sensibile ai costi è un sotto-campo di studio separato e il costo può essere definito in modo più ampio dell'errore di previsione o dell'errore di classificazione.

I costi ad esempio potrebbero essere:

- Costo degli errori di classificazione errata (o errori di previsione più in generale).
- Costo delle prove o della valutazione.
- Costo di etichettatura.
- Costo di calcolo o complessità computazionale.
- Costo dell'instabilità o della varianza noto come deriva concettuale.
- etc...

Come assegnare i costi dovuti ad una errata classificazione?

Partiamo dalla matrice di confusione e costruiamo una **matrice dei costi** tale che:

- $C(+, +) = C(-, -) = 0$ ovvero la corretta classificazione non ha alcun costo
- $C(+, -) > C(-, +)$ ovvero il costo di classificare erroneamente un negativo come un positivo (falso positivo) è maggiore del costo di classificare erroneamente un positivo come un negativo (falso negativo)

quindi usiamo la matrice dei costi per addestrare o valutare il modello di classificazione.

In alcuni domini problematici, la definizione della matrice dei costi potrebbe essere ovvia. In altri ambiti, definire la matrice dei costi potrebbe essere difficile. Inoltre, il costo potrebbe essere una complessa funzione multidimensionale

Un buon punto di partenza per le attività di classificazione sbilanciata consiste nell'assegnare i costi in base alla **distribuzione inversa delle classi**.

Ad esempio, potremmo avere un set di dati con un rapporto da 1 a 100 (1:100) tra esempi nella classe di minoranza e esempi nella classe di maggioranza. Questo rapporto può essere invertito e utilizzato come costo di errori di classificazione errata, dove il costo di un falso negativo è 100 e il costo di un falso positivo è 1.

Questa è un'**euristica** efficace per impostare i costi in generale, sebbene presuppone che la distribuzione delle classi osservata nei dati di addestramento sia rappresentativa del problema più ampio e sia appropriata per il metodo scelto in base ai costi utilizzato.

Cost-sensitive learning

In linea generale possiamo:

- utilizzare la matrice di costi per guidare il ri-campionamento
- utilizzare la matrice di costi per addestrare un modello di classificazione
- ripesare il modo in cui un modello assegna una classe ad un nuovo dato (se il modello è di tipo probabilistico) calcolando il rischio di un errore, $L(x, i) = \sum_j P(j|x)C(i, j)$. dove $L(x, i)$ è il rischio per un dato x di essere di classe i e $P(j|x)$ è calcolato tramite il teorema di Bayes.
- utilizzare una "ensemble" di modelli cost-insensitive, ognuno specifico per ogni classe

